ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN B

MÁY CHỦ TRUNG TÂM & CƠ SỞ DỮ LIỆU

Kiến trúc đa luồng TCP, động cơ quy tắc và giải quyết xung đột dữ liệu trên CSDL

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN B

(Đọc xong trang này trong 2 phút là bạn biết chính xác mình cần làm gì hôm nay)

1. BẠN LÀ AI?

Bạn là người xây dựng "bộ não" trung tâm của toàn bộ hệ thống: phần mềm máy chủ (**Central Server**) và cơ sở dữ liệu (**Database**).

2. BẠN LÀM GÌ?

- Mở cổng TCP 5000 để lắng nghe kết nối đồng thời từ nhiều máy sinh viên và máy quản trị.
- Tiếp nhận và lưu số liệu CPU, RAM vào cơ sở dữ liệu SQL Server.
- Tự động phát hiện khi máy sinh viên quá tải ($\geq$ 90%) hoặc tắt đột ngột ($>$ 20s không có nhịp tim).
- Xử lý yêu cầu từ Admin và tránh việc hai người cùng sửa một sự cố gây ghi đè dữ liệu.

3. BẠN KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo đo phần cứng máy trạm hay bắt sự kiện hệ thống (việc của **Thành viên A**).
- Bạn **không cần** lo thiết kế giao diện WinForms hay vẽ biểu đồ (việc của **Thành viên C**).
- Bạn **không cần** lo gom luồng Gateway hay chạy công cụ thử tải (việc của **Thành viên D**).

4. HÔM NAY BẮT ĐẦU Ở ĐÂU?

Xem `database/schema.sql` và `database/seed.sql` để biết vị trí phần CSDL; hai file hiện chưa được hiện thực. Sau đó mở file `src/LabGuard.Central/Program.cs` để viết vòng lặp lắng nghe TCP đầu tiên bằng `TcpListener`.

5. FILES BẠN CẦN MỞ TRƯỚC:

1. `START_HERE.md`: Tổng quan đồ án và cách khởi động hệ thống.
2. `database/schema.sql` và `database/seed.sql`: Nơi định nghĩa bảng CSDL trong tương lai (hiện là placeholder).
3. `src/LabGuard.Protocol/MessageTypes.cs`: Danh mục tên các loại thông điệp cần xử lý.
4. `src/LabGuard.Central/Program.cs`: Điểm khởi đầu của máy chủ trung tâm.

MỤC LỤC

1 Tổng quan vai trò của Thành viên B **1**

2 Bảy chặng phát triển máy chủ Central Server **2**
2.1 VIỆC TIẾP THEO: Mở cổng TCP 5000 và tiếp nhận kết nối đa luồng 2
2.2 Chặng 2: VIỆC TIẾP THEO: Tiếp nhận và giải mã khung tin từ Agent 3
2.3 Chặng 3: VIỆC TIẾP THEO: Lưu trữ số liệu máy trạm vào CSDL SQL Server . . 4
2.4 Chặng 4: VIỆC TIẾP THEO: Động cơ phát hiện máy tắt hoặc đứt mạng (Liveness Engine) 5
2.5 Chặng 5: VIỆC TIẾP THEO: Động cơ quy tắc phát hiện sự cố (Rule Engine) . . . 6
2.6 Chặng 6: VIỆC TIẾP THEO: Xử lý tranh chấp cập nhật bằng Khóa lạc quan (OCC) 7
2.7 Chặng 7: VIỆC TIẾP THEO: Viết kiểm thử tự động cho máy chủ Central 8

3 Năm bẫy lỗi lập trình cần tránh dành cho Thành viên B **9**

4 Bộ câu hỏi vấn đáp dành riêng cho Thành viên B **10**

1. Tổng quan vai trò của Thành viên B

TÓM TẮT 30 GIÂY

- Bạn phụ trách phần mềm máy chủ (**Central Server**) và cơ sở dữ liệu (**Database**).
- Máy chủ mở duy nhất cổng TCP 5000 để đón nhận cả máy trạm Agent và ứng dụng Admin.
- Đóng vai trò trọng tài: lưu trữ số liệu, phát hiện máy đứt mạng ($>$ 20s) và quá tải ($\geq$ 90%).
- Toàn bộ phân hệ Central hiện là khung dự án để bạn tự tay xây dựng.

Trong hệ thống LabGuard, bạn đóng vai trò là ”trái tim” và ”bộ não” điều phối toàn bộ luồng thông tin mạng. Mọi máy trạm Agent của sinh viên và phần mềm quản trị Admin đều phải kết nối về chương trình của bạn để gửi nhận dữ liệu:

KHÁI NIỆM MẠNG CỐT LÕI

Trọng trách cốt lõi của Thành viên B:

1. **Lập trình máy chủ Central Server (`src/LabGuard.Central/`):** Mở cổng TCP 5000 đón nhận kết nối đồng thời từ hàng chục máy tính mà không làm tắc nghẽn hệ thống.
2. **Quản trị Cơ sở dữ liệu (`database/`):** Lưu trữ bền vững số liệu tài nguyên, danh mục thiết bị và nhật ký sự cố vào hệ quản trị Microsoft SQL Server.
3. **Trọng tài tự động (Rule Engine & Liveness):** Tự động phát hiện khi một máy bị đứt mạng ($>$ 20 giây không có nhịp tim) hoặc bị quá tải CPU ($\geq$ 90%).

Hình này chỉ cần nhìn từ trên xuống dưới: từ luồng mạng tiếp nhận (cổng 5000), định tuyến thông điệp, đến động cơ kiểm tra và lưu vào cơ sở dữ liệu.

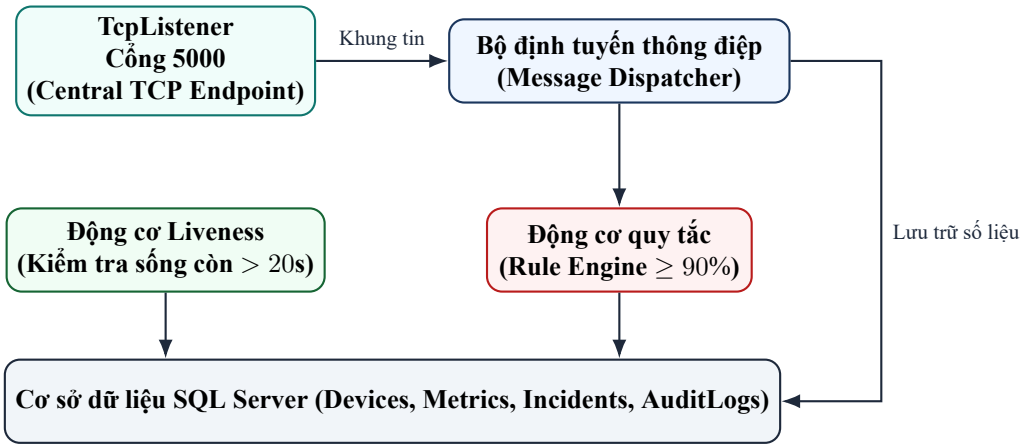


Figure 1: Kiến trúc nội bộ các thành phần trong máy chủ Central Server

Ý chính: Central Server là trung tâm tiếp nhận dữ liệu từ các máy trạm, chạy luật kiểm tra tự động và ghi nhận kết quả vào cơ sở dữ liệu SQL Server.

GHI CHÚ QUAN TRỌNG

Hiện trạng mã nguồn thực tế: Toàn bộ các tính năng của máy chủ Central hiện đang ở trạng thái **CHƯA LÀM / DỰ KIẾN**. Thư mục `src/LabGuard.Central/` hiện chỉ có file `Program.cs` rỗng. Bạn sẽ tự tay lập trình các tính năng theo lộ trình 7 chặng dưới đây.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Central Server là bên lắng nghe thụ động (Passive Open) trên cổng 5000.
- Mọi gói tin nhận được phải giải mã qua thư viện `LabGuard.Protocol` trước khi phân phối xử lý.

2. Bảy chặng phát triển máy chủ Central Server

TÓM TẮT 30 GIÂY

- Chặng 1–2: Mở cổng TCP 5000, nhận kết nối đa luồng và giải mã khung tin.
- Chặng 3: Kết nối CSDL SQL Server lưu trữ số liệu máy trạm an toàn.
- Chặng 4–5: Tự động phát hiện máy tắt (> 20s) và quá tải ($\geq 90\%$).
- Chặng 6–7: Giao tiếp với Admin bằng khóa lạc quan OCC và viết kiểm thử tự động.

2.1 VIỆC TIẾP THEO: Mở cổng TCP 5000 và tiếp nhận kết nối đa luồng

Chặng 1 — Mã tác vụ nội bộ: LG-B-001, LG-B-002

2.1.1 Tại sao cần phần này?

Central Server phải là bên mở cổng trước để sẵn sàng đón nhận các máy Agent kết nối vào. Một phòng máy có thể có 40–50 máy tính; nếu server chỉ phục vụ được 1 máy tại một thời điểm thì các máy còn lại sẽ bị treo kết nối ngay lập tức.

2.1.2 Khi chạy, chuyện gì xảy ra?

Central Server khởi tạo đối tượng `TcpListener` lắng nghe trên cổng 5000. Cả máy trạm Agent và ứng dụng Admin đều kết nối về cổng 5000 này. Mỗi khi có một kết nối mới, Central tách kết nối đó ra một luồng tác vụ riêng (`Task.Run`) để xử lý, đồng thời tiếp tục vòng lặp đón nhận máy tiếp theo.

2.1.3 Nhận diện loại máy kết nối (Handshake Discrimination)

Trên cùng cổng 5000, Central phân biệt đối tác dựa vào bản tin đầu tiên:

- Nếu bản tin đầu tiên là `HELLO_AGENT`: Đây là máy trạm sinh viên → Đưa vào danh sách giám sát và phản hồi `HELLO_ACK`.
- Nếu bản tin đầu tiên là `LOGIN`: Đây là ứng dụng Admin → Xác thực quyền quản trị và chuyển sang kênh điều khiển.

- Nếu là bản tin khác: Không hợp lệ → Phản hồi mã lỗi và đóng kết nối an toàn.

2.1.4 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Central/CentralServer.cs` và `ClientConnection.cs`.

2.1.5 Bạn cần làm những bước lớn nào?

1. Tạo và khởi động `TcpListener` trên cổng 5000.
2. Viết vòng lặp `AcceptTcpClientAsync`.
3. Đưa mỗi client vào một hàm xử lý bất đồng bộ riêng biệt.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Lắng nghe trên cổng 5000 với địa chỉ `IPAddress.Any` (chấp nhận mọi card mạng).
- Tách xử lý sang `Task.Run` để không chặn vòng lặp đón tiếp.

```
// Pseudocode minh hoa: Tiep nhan ket noi TCP da luong tren cong 5000
var listener = new TcpListener(IPAddress.Any, 5000);
listener.Start();
while (!ct.IsCancellationRequested)
{
    TcpClient client = await listener.AcceptTcpClientAsync(ct);
    _ = Task.Run(() => HandleClientAsync(client, ct), ct);
}
```

Listing 1: Pseudocode minh họa: Vòng lặp tiếp nhận kết nối TCP của Central Server

2.1.6 Làm sao thử?

Bật Central Server lên, mở PowerShell và gõ:

```
Test-NetConnection -ComputerName 127.0.0.1 -Port 5000
```

Nếu kết quả hiện `TcpTestSucceeded : True` nghĩa là máy chủ đã mở cổng thành công!

2.1.7 Khi nào coi là xong? (PASS Criteria)

- Lắng nghe đúng cổng TCP 5000 duy nhất.
- Tiếp nhận đồng thời nhiều kết nối cùng lúc mà không bị nghẽn.

2.2 Chặng 2: VIỆC TIẾP THEO: Tiếp nhận và giải mã khung tin từ Agent

Mã tác vụ nội bộ: LG-B-003

2.2.1 Tại sao cần phần này?

Dữ liệu truyền qua mạng là một dòng byte nhị phân liên tục. Central Server phải đọc chính xác 4 byte độ dài và tách đúng chuỗi JSON để chuyển thành đối tượng C# có thể xử lý được.

2.2.2 Khi chạy, chuyện gì xảy ra?

Với mỗi kết nối, Central chạy vòng lặp đọc khung tin qua `FrameCodec.ReadFrameAsync`. Sau đó giải mã thành `WireEnvelope` và gọi bộ kiểm tra `EnvelopeValidator.Validate` để bảo đảm bản tin an toàn và đúng chuẩn.

2.2.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Central/ClientConnection.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Đọc từng khung tin nhị phân qua `FrameCodec`.
- Giải mã và kiểm tra hợp lệ bằng `EnvelopeValidator` trước khi xử lý.

```
// Pseudocode minh hoa: Doc va giai ma khung tin tu ket noi TCP
while (!ct.IsCancellationRequested)
{
    byte[]? frame = await FrameCodec.ReadFrameAsync(stream, ct: ct);
    if (frame == null) break; // Client chu dong ngat ket noi
    var env = ProtocolSerializer.DeserializeEnvelope(frame);
    if (env != null && EnvelopeValidator.Validate(env).IsValid)
        await DispatchMessageAsync(env);
}
```

Listing 2: Pseudocode minh họa: Vòng lặp đọc và giải mã gói tin

2.2.4 Làm sao thử?

Cho Agent gửi bản tin `HELLO_AGENT`, Central Server phải in ra console:

```
[Central] Da nhan HELLO_AGENT tu DeviceId=PC-01
```

2.2.5 Khi nào coi là xong? (PASS Criteria)

- Đọc chính xác từng khung tin nhị phân qua `FrameCodec`.
- Xác thực chặt chẽ 6 trường bắt buộc của `WireEnvelope`.
- Giải phóng tài nguyên socket an toàn khi client ngắt kết nối.

2.3 Chặng 3: VIỆC TIẾP THEO: Lưu trữ số liệu máy trạm vào CSDL SQL Server

Mã tác vụ nội bộ: LG-B-004

2.3.1 Tại sao cần phần này?

Nếu không lưu vào cơ sở dữ liệu, mọi thông số CPU/RAM và lịch sử sự cố sẽ biến mất hoàn toàn ngay khi tắt máy chủ. CSDL giúp lưu vết lịch sử phục vụ cho việc thống kê và báo cáo.

2.3.2 Khi chạy, chuyện gì xảy ra?

Khi nhận gói tin `METRIC_BATCH`, máy chủ trích xuất các mẫu đo tài nguyên và thực thi câu lệnh SQL có tham số để lưu trữ vào bảng `Metrics`.

2.3.3 Bạn sẽ đụng file nào?

Tạo mới hoặc hoàn thiện: `database/schema.sql`, `database/seed.sql` và `src/LabGuard.Central/Databa`

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Luôn dùng Parameterized Query (`@DeviceId`, `@MetricValue`) để phòng chống tấn công SQL Injection.
- Thực thi bất đồng bộ bằng `ExecuteNonQueryAsync()`.

```
// Pseudocode minh hoa: Ghi so lieu vao SQL Server an toan
const string sql = "INSERT INTO Metrics (DeviceId, MetricName, MetricValue)
    VALUES (@Dev, @Name, @Val);";
using var conn = new SqlConnection(_connString);
await conn.OpenAsync();
using var cmd = new SqlCommand(sql, conn);
cmd.Parameters.AddWithValue("@Dev", deviceId);
cmd.Parameters.AddWithValue("@Name", sample.MetricName);
cmd.Parameters.AddWithValue("@Val", sample.MetricValue);
await cmd.ExecuteNonQueryAsync();
```

Listing 3: Pseudocode minh họa: Thao tác ghi số liệu đo đạc vào CSDL

2.3.4 Làm sao thử?

Dùng SQL Server Management Studio mở bảng `Metrics` và kiểm tra các dòng dữ liệu mới xuất hiện.

2.3.5 Khi nào coi là xong? (PASS Criteria)

- 100% truy vấn sử dụng Parameterized Query (tuyệt đối không cộng chuỗi SQL).
- Thao tác CSDL hoàn toàn bất đồng bộ (`OpenAsync`, `ExecuteNonQueryAsync`).

2.4 Chặng 4: VIỆC TIẾP THEO: Động cơ phát hiện máy tắt hoặc đứt mạng (Liveness Engine)

Mã tác vụ nội bộ: LG-B-005

2.4.1 Tại sao cần phần này?

Khi sinh viên rút dây mạng hoặc máy bị cúp điện đột ngột, socket phía server không hề nhận được gói tin báo tắt. Central Server phải có một cơ chế tự động đếm thời gian để phát hiện máy nào đã mất kết nối.

2.4.2 Khi chạy, chuyện gì xảy ra?

Mỗi khi nhận gói `HEARTBEAT`, Central cập nhật mốc thời gian nhận tin cuối cùng bằng đồng hồ đơn điệu (`Stopwatch.GetTimestamp`). Một tác vụ nền quét định kỳ mỗi 5 giây; nếu một máy im lặng quá 20 giây (> 20s), Central chuyển trạng thái máy sang `OFFLINE` và tạo sự cố `DEVICE_OFFLINE`.

2.4.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Central/LivenessMonitor.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Dùng `Stopwatch.GetElapsedTime` đo thời gian đơn điệu, không sợ bị nhảy giờ hệ thống.
- Ngưỡng 20 giây (> 20s) là thời điểm chính thức kích hoạt sự cố `DEVICE_OFFLINE`.

```
// Pseudocode minh hoa: Kiem tra song con dinh ky moi 5 giay
using var timer = new PeriodicTimer(TimeSpan.FromSeconds(5));
while (await timer.WaitForNextTickAsync(ct))
{
    long now = Stopwatch.GetTimestamp();
    foreach (var session in _registry.GetActiveSessions())
    {
        if (Stopwatch.GetElapsedTime(session.LastHeartbeat, now).TotalSeconds >
            20)
        {
            session.Status = "OFFLINE";
            await RaiseIncidentAsync(session.DeviceId, "DEVICE_OFFLINE",
                "CRITICAL");
        }
    }
}
```

Listing 4: Pseudocode minh họa: Quét định kỳ phát hiện máy quá hạn nhịp tim

2.4.4 Làm sao thử?

Cho Agent kết nối vào Central rồi bấm `Ctrl+C` tắt ngang Agent: đếm đúng hơn 20 giây (> 20s), màn hình Central phải in thông báo phát hiện máy mất kết nối!

2.4.5 Khi nào coi là xong? (PASS Criteria)

- Dùng đồng hồ đơn điệu (`Stopwatch`) tránh lỗi chỉnh giờ hệ thống.
- Nhận diện chính xác trạng thái `OFFLINE` sau 20 giây im lặng.
- Tự động tạo sự cố chuẩn `DEVICE_OFFLINE`.

2.5 Chặng 5: VIỆC TIẾP THEO: Động cơ quy tắc phát hiện sự cố (Rule Engine)

Mã tác vụ nội bộ: LG-B-006

2.5.1 Tại sao cần phần này?

Nếu sinh viên vừa bấm mở một phần mềm nặng khiến CPU vọt lên 90% trong 1 giây rồi hạ xuống thì đó chưa phải sự cố. Nhưng nếu CPU duy trì $\geq 90\%$ liên tục nhiều chu kỳ thì chắc chắn máy tính đang bị treo hoặc chạy vòng lặp vô tận.

2.5.2 Khi chạy, chuyện gì xảy ra?

Động cơ quy tắc duy trì một cửa sổ trượt lưu 3 mẫu đo gần nhất của từng máy trạm:

- **Kích hoạt sự cố:** Nếu CPU $\geq 90\%$ trong 3 mẫu đo liên tiếp, Central tạo sự cố `HIGH_CPU`.
- **Tự động phục hồi:** Nếu sau đó CPU hạ xuống $\leq 75\%$ trong 3 mẫu liên tiếp, Central tự động đóng sự cố.

2.5.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Central/RuleEngine.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Dùng hàng đợi 3 phần tử để lọc nhiễu cảnh báo giả (Debouncing).
- Tự động đóng sự cố khi chỉ số đo hạ về mức an toàn $\leq 75\%$.

```
// Pseudocode minh hoa: Luat phat hien qua tai CPU 3 mau lien tiep
var window = GetCpuWindow(deviceId);
window.Enqueue(cpuValue);
if (window.Count > 3) window.Dequeue();

if (window.Count == 3 && window.All(v => v >= 90.0))
    await RaiseIncidentAsync(deviceId, "HIGH_CPU", "WARNING");
else if (window.Count == 3 && window.All(v => v <= 75.0))
    await AutoResolveIncidentAsync(deviceId, "HIGH_CPU");
```

Listing 5: Pseudocode minh họa: Phát hiện quá tải với cửa sổ trượt

2.5.4 Làm sao thử?

Gửi 3 mẫu CPU 95% liên tiếp: Central phải báo động có sự cố `HIGH_CPU`. Gửi tiếp 3 mẫu CPU 50%: sự cố được tự động chuyển sang trạng thái giải quyết!

2.5.5 Khi nào coi là xong? (PASS Criteria)

- Áp dụng cửa sổ trượt 3 mẫu liên tiếp để lọc nhiễu cảnh báo giả.
- Tự động đóng sự cố khi giá trị đo hạ xuống dưới ngưỡng phục hồi (75%).

2.6 Chặng 6: VIỆC TIẾP THEO: Xử lý tranh chấp cập nhật bằng Khóa lạc quan (OCC)

Mã tác vụ nội bộ: LG-B-007

2.6.1 Tại sao cần phần này?

Phòng thực hành có thể có nhiều người quản trị cùng trực ban. Nếu Admin 1 và Admin 2 cùng nhìn thấy sự cố #10 và cùng bấm nút "Tiếp nhận" tại một thời điểm, máy chủ phải đảm bảo dữ liệu không bị ghi đè hỗn loạn.

2.6.2 Khi chạy, chuyện gì xảy ra?

Hệ thống sử dụng cơ chế kiểm soát tương tranh lạc quan (**Optimistic Concurrency Control — OCC**) dựa trên phiên bản dữ liệu (`ExpectedRowVersion`):

- Khi Admin mở xem sự cố #10, phiên bản bản ghi đang là 1.
- Admin 1 gửi lệnh tiếp nhận kèm `ExpectedRowVersion = 1`. Central cập nhật thành công và tăng phiên bản lên 2, phản hồi mã `200 OK`.
- Admin 2 cũng gửi lệnh tiếp nhận kèm `ExpectedRowVersion = 1`. Central thấy phiên bản trong CSDL lúc này đã là 2, không khớp với số 1 nữa → Từ chối cập nhật và trả về mã lỗi `409 Conflict` để Admin 2 tải lại dữ liệu mới nhất.

2.6.3 Bạn sẽ đụng file nào?

Tạo mới file: `src/LabGuard.Central/AdminService.cs`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Câu lệnh SQL kiểm tra đồng thời cả `Id` và `RowVersion = @ExpectedVer`.
- Nếu số dòng cập nhật được bằng 0, nghĩa là đã bị sửa trước đó → Báo lỗi 409 Conflict.

```
// Pseudocode minh hoa: Cap nhat su co co kiem tra phien ban (OCC)
const string sql = "UPDATE Incidents SET WorkflowState = 'ACKNOWLEDGED',
    RowVersion = @NewVer " +
                   "WHERE Id = @Id AND RowVersion = @ExpectedVer;";
using var cmd = new SqlCommand(sql, conn);
cmd.Parameters.AddWithValue("@Id", id);
cmd.Parameters.AddWithValue("@ExpectedVer", expectedVersion);
int rows = await cmd.ExecuteNonQueryAsync();
return rows > 0 ? 200 : 409; // 200 OK hoac 409 Conflict
```

Listing 6: Pseudocode minh họa: Cập nhật sự cố với khóa lạc quan OCC

2.6.4 Khi nào coi là xong? (PASS Criteria)

- Phát hiện chính xác xung đột phiên bản khi nhiều người thao tác đồng thời.
- Trả về mã lỗi 409 Conflict chuẩn giao thức mạng khi phát hiện mâu thuẫn dữ liệu.

2.7 Chặng 7: VIỆC TIẾP THEO: Viết kiểm thử tự động cho máy chủ Central

Mã tác vụ nội bộ: LG-B-008, LG-B-009

2.7.1 Tại sao cần phần này?

Central Server là trung tâm của toàn bộ hệ thống. Viết kiểm thử tự động giúp xác nhận động cơ quy tắc và bộ đếm nhịp tim hoạt động chính xác 100% trước khi nộp bài.

2.7.2 Bạn sẽ đụng file nào?

Tạo các tệp kiểm thử trong: `tests/LabGuard.Central.Tests/`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Gửi lần lượt 3 mẫu CPU quá tải để kiểm tra RuleEngine kích hoạt sự cố.
- Khẳng định sự cố được tạo đúng lúc mẫu thứ 3 xuất hiện.

```
// Vi du rut gon: Kiem tra RuleEngine kich hoat sau 3 mau CPU >= 90%
[Fact]
public async Task RuleEngine_When3SamplesAbove90_RaisesIncident()
{
    var engine = new RuleEngine();
    await engine.EvaluateCpuAsync("PC-01", 92.0);
    await engine.EvaluateCpuAsync("PC-01", 94.0);
    await engine.EvaluateCpuAsync("PC-01", 95.0); // Mau thu 3 phai bao HIGH_CPU
}
```

Listing 7: Ví dụ rút gọn: Kiểm thử đơn vị cho động cơ quy tắc RuleEngine

2.7.3 Khi nào coi là xong? (PASS Criteria)

- Lệnh `dotnet test tests/LabGuard.Central.Tests` chạy hoàn tất thành công.
- 100% các ca kiểm thử đạt kết quả Passed (Failed: 0).

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Central Server điều phối toàn bộ luồng mạng qua cổng 5000.
- Sử dụng Parameterized Query trên CSDL và cơ chế OCC khi cập nhật sự cố.

3. Năm bẫy lỗi lập trình cần tránh dành cho Thành viên B

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Đồng bộ hóa luồng mạng bằng Thread.Sleep()
Tuyệt đối không dùng `Thread.Sleep()` trong các tác vụ mạng của Central Server. Lệnh này sẽ đóng băng toàn bộ luồng xử lý, khiến tất cả các máy trạm khác kết nối vào đều bị treo cứng. Luôn dùng `await Task.Delay()`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- `Thread.Sleep` làm chặn đứng luồng của ThreadPool.
- `await Task.Delay` giải phóng luồng trong lúc chờ, không làm ảnh hưởng các máy khác.

```
// [SAI] Dong bang luong cua he thong:
Thread.Sleep(5000); // Cam tuyet doi!

// [DUNG] Giai phong luong de tiep tuc phuc vu may khac:
await Task.Delay(5000, token); // Async non-blocking
```

Listing 8: So sánh Bẫy 1: Thread.Sleep vs Task.Delay

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Dùng Dictionary thường thay vì ConcurrentDictionary
Central Server có hàng chục kết nối đọc ghi danh sách máy trạm cùng lúc. Nếu dùng `Dictionary` thông thường sẽ ném ngoại lệ `InvalidOperationException` và làm sập máy chủ. Luôn dùng `ConcurrentDictionary`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Cộng chuỗi SQL gây lỗ hổng SQL Injection
Không bao giờ viết: `"WHERE DeviceId = '" + id + "'"`. Kẻ xấu có thể gửi chuỗi độc hại phá hủy toàn bộ cơ sở dữ liệu. Luôn dùng tham số hóa `cmd.Parameters.AddWithValue`.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Cộng chuỗi trực tiếp tạo lỗ hổng SQL Injection nghiêm trọng.
- Dùng tham số hóa `@Id` giúp CSDL tự động làm sạch dữ liệu đầu vào.

```
// [SAI] Nguy co bi tan cong pha huy CSDL:
string sql = "SELECT * FROM Devices WHERE DeviceId = '" + inputId + "'";

// [DUNG] An toan tuyet doi voi tham so hoa:
string sql = "SELECT * FROM Devices WHERE DeviceId = @Id";
cmd.Parameters.AddWithValue("@Id", inputId);
```

Listing 9: So sánh Bẫy 3: Cộng chuỗi SQL vs Parameterized Query

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Tính thời gian sống còn bằng DateTime.UtcNow
Nếu đồng hồ máy chủ bị chỉnh giờ hoặc nhảy mạng NTP lùi lại, thuật toán Liveness sẽ tính ra số âm hoặc ngắt kết nối oan máy trạm. Luôn dùng `Stopwatch.GetTimestamp()` để đo thời gian đơn điệu.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Quên đóng kết nối CSDL (Connection Leak)
Mỗi khi tạo đối tượng `SqlConnection`, nếu không đặt trong khối lệnh `using`, sau một thời gian kết nối CSDL sẽ bị cạn kiệt (*Connection Pool Exhausted*) và máy chủ sẽ dừng hoạt động.

4. Bộ câu hỏi vấn đáp dành riêng cho Thành viên B

Dưới đây là 20 câu hỏi trọng tâm thường gặp nhất khi bảo vệ đồ án phần máy chủ trung tâm:

Câu 1: Máy chủ Central lắng nghe trên cổng nào?
Trả lời: Máy chủ lắng nghe trên cổng TCP 5000 (Central TCP Endpoint), đóng vai trò là điểm tiếp nhận duy nhất cho toàn bộ hệ thống phòng máy. Cả máy trạm Agent và ứng dụng Admin đều kết nối về cổng 5000 này.

Câu 2: Tại sao lại sử dụng chung cổng 5000 mà không mở nhiều cổng phức tạp?
Trả lời: Sử dụng chung cổng 5000 giúp cấu hình mạng tối giản, tương thích hoàn hảo với chính sách tường lửa phòng máy. Central Server phân biệt vai trò của đối tác (Agent hay Admin) thông qua bản tin bắt tay đầu tiên (`HELLO_AGENT` hoặc `LOGIN`) để điều phối luồng xử lý tương ứng.

Câu 3: Làm thế nào máy chủ phục vụ được 50 máy trạm cùng một lúc mà không bị treo?
Trả lời: Sử dụng mô hình lập trình bất đồng bộ (`async/await`) kết hợp cơ chế I/O Completion Ports (IOCP) của .NET và Windows, cho phép một số lượng luồng nhỏ trong ThreadPool có thể xử lý hàng ngàn kết nối mạng đồng thời.

Câu 4: Khi một máy trạm ngắt kết nối đột ngột, server phát hiện bằng cách nào?
Trả lời: Nếu máy trạm bị ngắt vật lý (rút dây mạng), server không nhận được gói tin báo tắt. Server sẽ phát hiện thông qua luồng quét định kỳ của `LivenessMonitor`: nếu máy trạm im lặng hơn 20 giây (> 20s) thì server chính thức đánh dấu trạng thái `OFFLINE`.

Câu 5: Tại sao em lại dùng Stopwatch.GetTimestamp() thay vì DateTime.UtcNow?
Trả lời: `DateTime.UtcNow` là đồng hồ thời gian thực tế có thể bị nhảy cóc khi hệ điều hành đồng bộ giờ NTP hoặc chỉnh giờ. `Stopwatch.GetTimestamp()` là đồng hồ đơn điệu (Monotonic Clock) luôn tăng đều, bảo đảm việc tính khoảng cách thời gian chính xác tuyệt đối.

Câu 6: Thuật toán Khóa lạc quan (OCC) trên cơ sở dữ liệu hoạt động ra sao?
Trả lời: CSDL sử dụng cột `RowVersion` trong bảng `Incidents`. Khi cập nhật, câu lệnh SQL kiểm tra `WHERE Id = @Id AND RowVersion = @ExpectedVersion`. Nếu số hàng bị ảnh hưởng bằng 1 thì thành công; nếu bằng 0 nghĩa là đã có người sửa trước, server trả về mã lỗi 409 Conflict.

Câu 7: Quy tắc cảnh báo CPU quá tải hoạt động như thế nào?
Trả lời: Động cơ quy tắc yêu cầu mức tải CPU $\geq 90\%$ trong 3 mẫu đo liên tiếp để kích hoạt sự cố `HIGH_CPU`, và yêu cầu CPU $\leq 75\%$ liên tục trong 3 mẫu để tự động giải quyết sự cố.

Câu 8: Cơ sở dữ liệu của em hiện tại đã hiện thực chưa và dự kiến gồm những gì?
Trả lời: CSDL hiện tại chưa được hiện thực. Số bảng cuối cùng sẽ được chốt khi thực hiện nhiệm vụ thiết kế schema (Task B-005). Các thực thể dự kiến gồm: `Devices`, `Metrics`, `Incidents`, và `AuditLogs`.

Câu 9: Làm thế nào để phòng tránh tấn công SQL Injection?
Trả lời: Luôn sử dụng truy vấn tham số hóa (Parameterized Queries) thông qua `SqlCommand.Paramete` tuyệt đối không bao giờ cộng chuỗi lệnh SQL.

Câu 10: Tại sao phải dùng ConcurrentDictionary để lưu danh sách kết nối?
Trả lời: Vì `Dictionary` thông thường không an toàn đa luồng. Khi nhiều luồng cùng thêm, xóa hoặc đọc kết nối cùng lúc sẽ ném ngoại lệ `InvalidOperationException`. `ConcurrentDictionary` giải quyết an toàn vấn đề này.

Câu 11: Khi nhận được một khung tin nhị phân, Central Server làm những bước gì?
Trả lời: 1. Đọc đủ khung tin bằng `FrameCodec`; 2. Giải mã JSON thành `WireEnvelope`; 3. Kiểm tra tính hợp lệ bằng `EnvelopeValidator`; 4. Điều phối thông điệp tới bộ phận xử lý tương ứng.

Câu 12: Nếu hai Admin cùng xác nhận một sự cố cùng lúc thì chuyện gì xảy ra?
Trả lời: Nhờ cơ chế khóa lạc quan (OCC), người gửi lệnh đến trước sẽ cập nhật thành công

(nhận `200 OK`), người gửi đến sau sẽ bị từ chối (nhận `409 Conflict`) và màn hình sẽ yêu cầu tải lại dữ liệu mới nhất.

Câu 13: Làm thế nào để giới hạn số lượng kết nối tối đa vào Server?
Trả lời: Bằng cách kiểm tra số lượng phần tử trong `ConcurrentDictionary`. Nếu vượt quá ngưỡng (ví dụ 100 kết nối), máy chủ từ chối kết nối mới bằng cách đóng socket ngay lập tức.

Câu 14: Bảng Metrics có cần tạo Index không? Tại sao?
Trả lời: Cần tạo chỉ mục không phân cụm (Non-clustered Index) trên 2 cột `(DeviceId, RecordedAtUtc DESC)` để tối ưu hóa tốc độ truy vấn biểu đồ lịch sử và tìm kiếm số liệu mới nhất của từng máy trạm.

Câu 15: Nếu máy chủ SQL Server bị dừng đột ngột thì Central xử lý thế nào?
Trả lời: Central Server duy trì kết nối TCP nếu có thể; nhịp tim `HEARTBEAT` và liveness vẫn hoạt động trên RAM. Central **không** duy trì hàng đợi đệm số liệu thứ hai trên RAM. Khi nhận `METRIC_BATCH` lúc CSDL gián đoạn, server không commit và không gửi `BATCH_ACK`; phía Agent giữ quyền sở hữu số liệu qua `ReplayBuffer` và sẽ gửi bù sau khi CSDL phục hồi. Các lệnh ghi của Admin sẽ nhận mã lỗi `ERR_DATABASE_UNAVAILABLE`.

Câu 16: Giới hạn kích thước gói tin mà Server chấp nhận là bao nhiêu?
Trả lời: Là 262,144 byte (256 KiB), quy định chặt chẽ trong `FrameCodec.MaxPayloadSize`.

Câu 17: Khi tắt Server thì làm sao để các kết nối không bị lỗi dữ liệu dở dang?
Trả lời: Kích hoạt `CancellationTokenSource`, ngừng đón nhận kết nối mới, đợi các tác vụ đọc ghi hiện tại hoàn tất (`Graceful Shutdown`) rồi mới đóng các socket.

Câu 18: Central Server lưu nhật ký hành vi quản trị vào đâu?
Trả lời: Lưu vào bảng `AuditLogs`, ghi lại thông tin quản trị viên, hành động thực hiện, thiết bị đích và thời điểm thao tác.

Câu 19: Hiện tại mã nguồn của Central Server đã làm được gì?
Trả lời: Hiện tại mã nguồn Central đang ở giai đoạn phân công nhiệm vụ (CHƯA LÀM / DỰ KIẾN). Toàn bộ phần việc này em sẽ tự tay lập trình và tích hợp với thư viện Protocol đã hoàn thành của nhóm.

Câu 20: Em phối hợp với Thành viên A và C như thế nào?
Trả lời: Em sử dụng thư viện chung `LabGuard.Protocol` của bạn A để nhận gói tin từ Agent; đồng thời em cung cấp điểm tiếp nhận TCP trên cổng 5000 để bạn C kết nối giao diện WinForms vào lấy số liệu.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN B —